Greedy Algorithm: Huffman Coding

Design and Analysis of Algorithms

Dhannya SM

Problem Statement

- Huffman coding is a lossless data compression algorithm used to minimize the total number of bits required to represent a set of symbols based on their frequencies. It was developed by David A. Huffman in 1952.
- Consider the string: "hello huffman"
- If we use 4 bits for each character, then the string requires $13 \times 4 = 52$ bits.

Problem Statement

- Huffman coding is a lossless data compression algorithm used to minimize the total number of bits required to represent a set of symbols based on their frequencies. It was developed by David A. Huffman in 1952.
- Consider the string: "hello huffman"
- If we use 4 bits for each character, then the string requires $13 \times 4 = 52$ bits.
- Instead, if we use number of bits proportionate to the frequency, we need fewer number of bits to represent this string.

Problem Statement

- Huffman coding is a lossless data compression algorithm used to minimize the total number of bits required to represent a set of symbols based on their frequencies. It was developed by David A. Huffman in 1952.
- Consider the string: "hello huffman"
- If we use 4 bits for each character, then the string requires $13 \times 4 = 52$ bits.
- Instead, if we use number of bits proportionate to the frequency, we need fewer number of bits to represent this string.
- The frequency of each character is given below:
  h:2, e:1, l:2, o:1, (space):1, u:1, f:2, m:1, a:1, n:1

Problem Statement

- Huffman coding is a lossless data compression algorithm used to minimize the total number of bits required to represent a set of symbols based on their frequencies. It was developed by David A. Huffman in 1952.
- Consider the string: "hello huffman"
- If we use 4 bits for each character, then the string requires $13 \times 4 = 52$ bits.
- Instead, if we use number of bits proportionate to the frequency, we need fewer number of bits to represent this string.
- The frequency of each character is given below:
  h:2, e:1, l:2, o:1, (space):1, u:1, f:2, m:1, a:1, n:1
- Suppose we represent each character using the bits shown below:
  h: 00, e: 1100, l: 10, o: 1101, (space): 1110, u: 1111, f: 01, m: 1011, a: 1010, n: 1001
- Then the total number of bits needed is:
  $2 \times 2 + 1 \times 4 + 2 \times 2 + 1 \times 4 + 1 \times 4 + 1 \times 4 + 2 \times 2 + 1 \times 4 + 1 \times 4 + 1 \times 4 = 40$
- How do we find this representation?

Construct the Huffman tree

- Count how often each piece of data occurs.
- Build a binary tree, starting with the nodes with the lowest count. The new parent node has the combined count of its child nodes.
- The edge from a parent gets '0' for the left child, and '1' for the edge to the right child.
- In the finished binary tree, follow the edges from the root node, adding '0' or '1' for each branch, to find the new Huffman code for each piece of data.
- Create the Huffman code by converting the data, piece-by-piece, into a binary code using the binary tree.

Example: BCCABBDDAECCBBAEDDCC

Count frequency of each character

Example: BCCABBDDAECCBBAEDDCC

Count frequency of each character

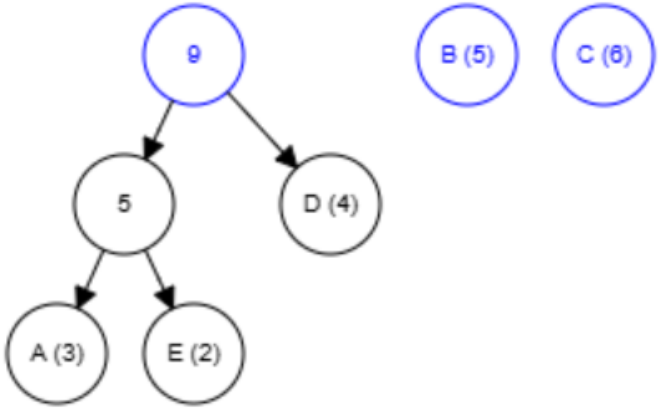
Build a binary tree, starting with nodes with lowest count.
New parent node has the combined count of its child nodes.

Example: BCCABBDDAECCBBAEDDCC

Count frequency of each character

Build a binary tree, starting with nodes with lowest count.
New parent node has the combined count of its child nodes.

Continue building the binary tree

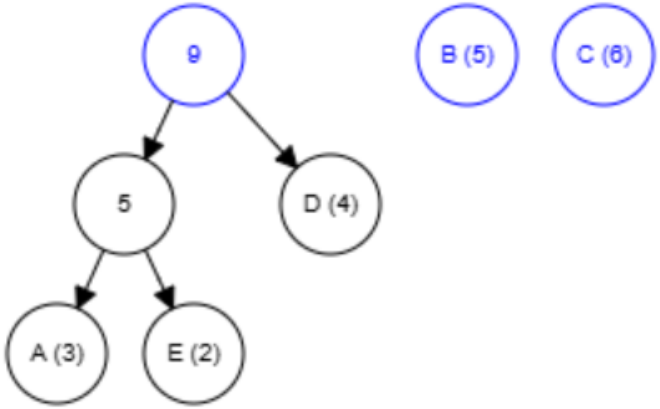


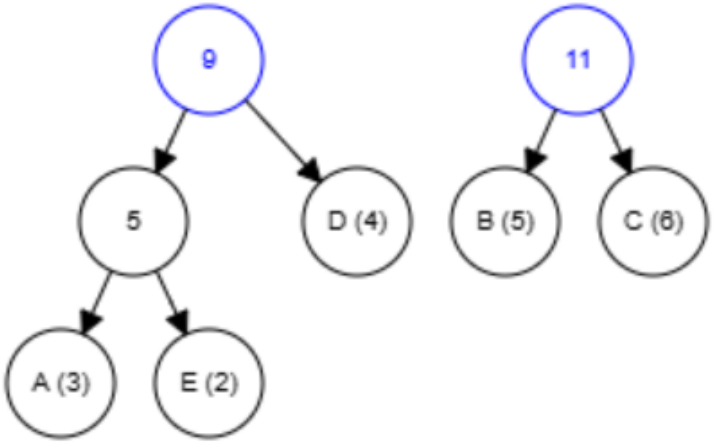
Example: BCCABBDDAECCBBAEDDCC

Count frequency of each character

Build a binary tree, starting with nodes with lowest count.
New parent node has the combined count of its child nodes.

Continue building the binary tree



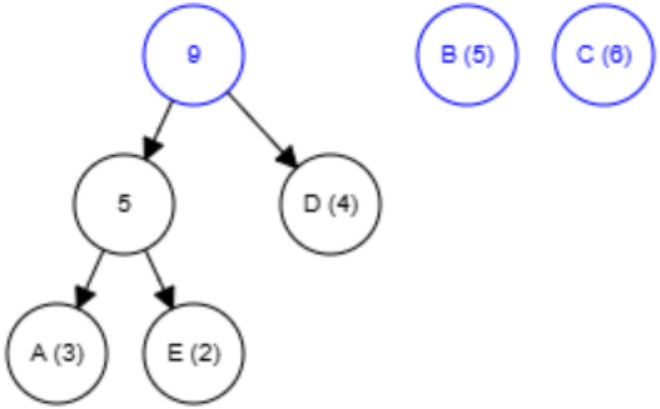


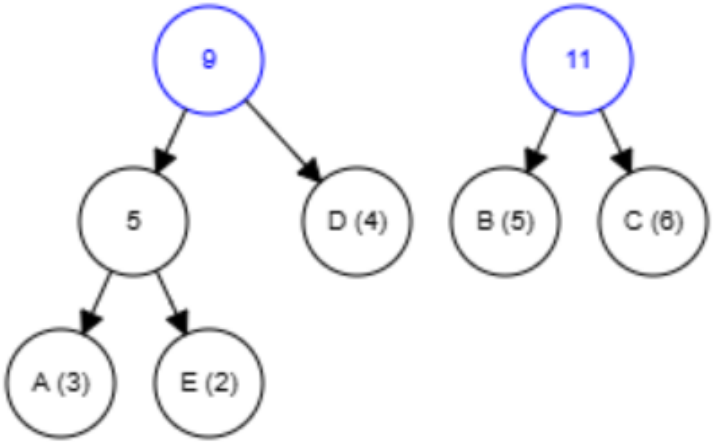
Example: BCCABBDDAECCBBAEDDCC

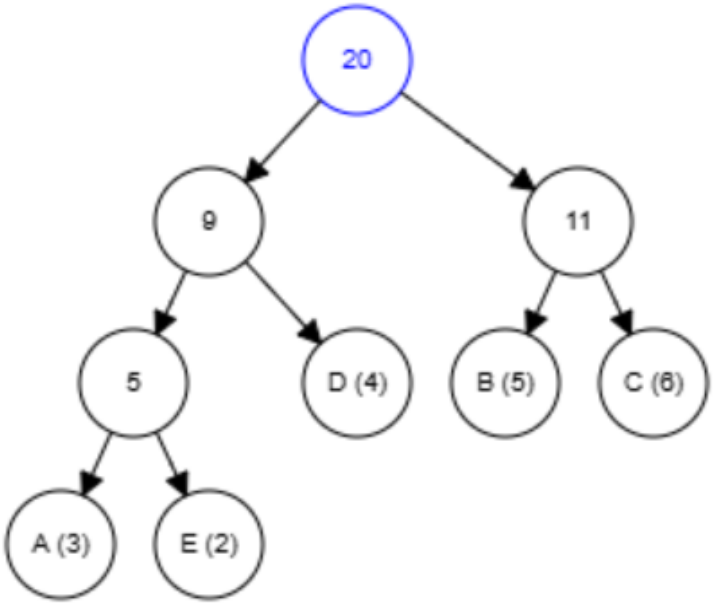
Count frequency of each character

Build a binary tree, starting with nodes with lowest count.
New parent node has the combined count of its child nodes.

Continue building the binary tree

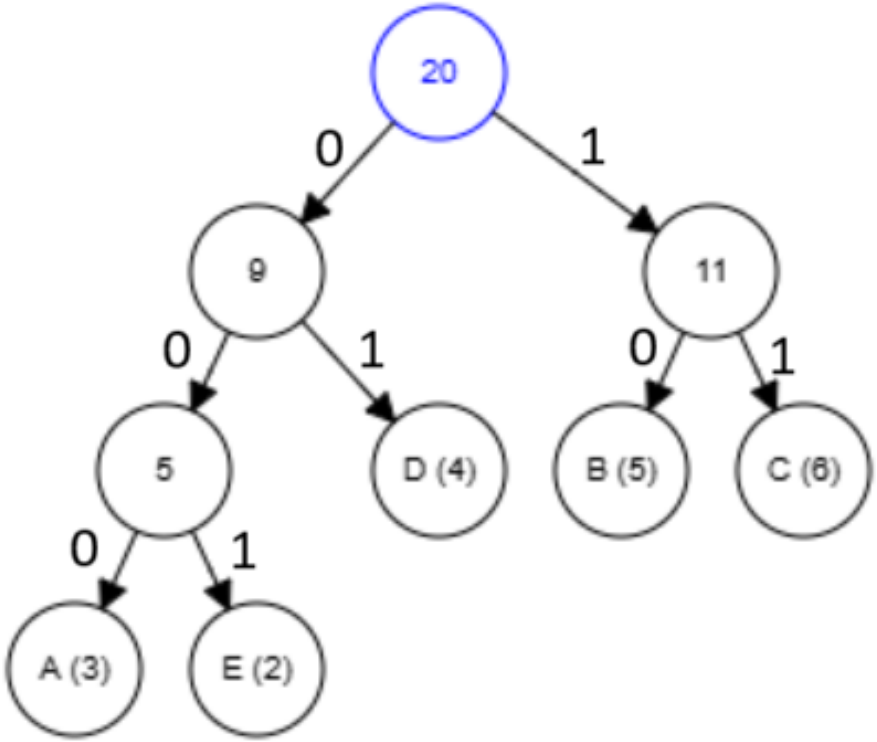






Example: BCCABBDDAECCBBAEDDCC (cont.)

The edge from a parent gets '0' for the left child, and '1' for the edge to the right child.



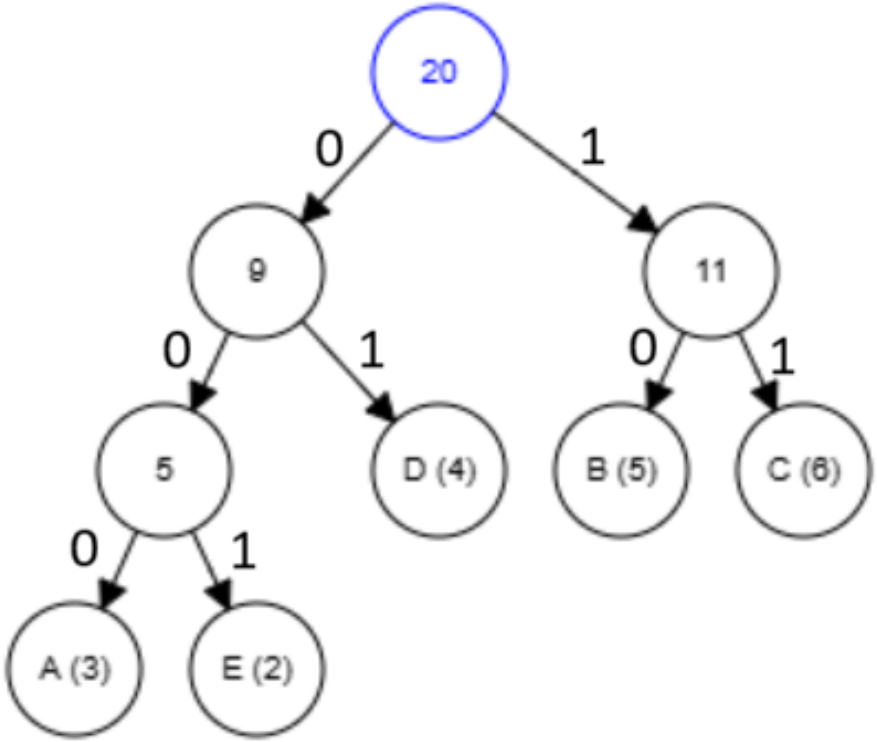
Follow the edges from the root node, adding '0' or '1' for each branch, to find the new Huffman code for each character.

Character	Code
A	000
B	10
C	11
D	01
E	001

Note that if we had written E before A in the tree, A would have got 001 and E would have 000.

Example: BCCABBDDAECCBBAEDDCC (cont.)

The edge from a parent gets '0' for the left child, and '1' for the edge to the right child.



Follow the edges from the root node, adding '0' or '1' for each branch, to find the new Huffman code for each character.

Character	Code
A	000
B	10
C	11
D	01
E	001

Note that if we had written E before A in the tree, A would have got 001 and E would have 000.

Final Huffman code for BCCABBDDAECCBBAEDDCC will be:
101111000101001010000011111101000000101011111

which needs only 45 bits as opposed to 160 bits (20x8) had we used simple ASCII code for each character.

Summary

- Definition
- Construction
- Example